STEP SEM-3 Coding Club | Category C Coding Assignment

Object-Oriented Programming: Abstract Classes, Interfaces, and Design Principles — Complete Solutions

---

Problem 1: Movie Ticket Counter

Implements an abstract base class Ticket with shared convenience fee logic and concrete subclasses for REGULAR, PREMIUM, and RECLINER seats.

Java Implementation:

```
import java.util.Scanner;

abstract class Ticket {
    protected int count;
    protected static final double CONVENIENCE_FEE = 20.0;

    public Ticket(int count) {
        this.count = count;
    }

    public abstract double getUnitPrice();

    public double calculateTotal() {
        return (getUnitPrice() + CONVENIENCE_FEE) * count;
    }
}

class RegularTicket extends Ticket {
    public RegularTicket(int count) { super(count); }
    @Override public double getUnitPrice() { return 150.0; }
}

class PremiumTicket extends Ticket {
    public PremiumTicket(int count) { super(count); }
    @Override public double getUnitPrice() { return 250.0; }
}

class ReclinerTicket extends Ticket {
    public ReclinerTicket(int count) { super(count); }
    @Override public double getUnitPrice() { return 400.0; }
}

public class MovieTicketCounter {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        if (!scanner.hasNextInt()) return;
        int n = scanner.nextInt();
        double grandTotal = 0.0;

        for (int i = 0; i < n; i++) {
            String type = scanner.next();
            int count = scanner.nextInt();
            Ticket ticket = null;

            switch (type.toUpperCase()) {
                case "REGULAR": ticket = new RegularTicket(count); break;
                case "PREMIUM": ticket = new PremiumTicket(count); break;
                case "RECLINER": ticket = new ReclinerTicket(count); break;
            }

            if (ticket != null) {
                double amount = ticket.calculateTotal();
                grandTotal += amount;
```

```
                System.out.printf("%s: %.2f
", type.toUpperCase(), amount);
            }
        }
        System.out.printf("Total: %.2f
", grandTotal);
        scanner.close();
    }
}
```

Problem 2: Parcel Shipping Desk

Demonstrates interface segregation using an Insurable interface implemented only by Express and Fragile parcels, keeping standard parcels separate.

Java Implementation:

```
import java.util.Scanner;

interface Insurable {
    double calculateInsurance(double declaredValue);
}

abstract class Parcel {
    protected double weightKg;
    protected double declaredValue;

    public Parcel(double weightKg, double declaredValue) {
        this.weightKg = weightKg;
        this.declaredValue = declaredValue;
    }

    public abstract double calculateCharge();
}

class StandardParcel extends Parcel {
    public StandardParcel(double w, double v) { super(w, v); }
    @Override public double calculateCharge() { return 40.0 + (10.0 * weightKg); }
}

class ExpressParcel extends Parcel implements Insurable {
    public ExpressParcel(double w, double v) { super(w, v); }
    @Override public double calculateCharge() { return 80.0 + (15.0 * weightKg); }
    @Override public double calculateInsurance(double declaredValue) { return 0.02 * declaredValue; }
}

class FragileParcel extends Parcel implements Insurable {
    public FragileParcel(double w, double v) { super(w, v); }
    @Override public double calculateCharge() { return 40.0 + (10.0 * weightKg) + 50.0; }
    @Override public double calculateInsurance(double declaredValue) { return 0.02 * declaredValue; }
}

public class ParcelShippingDesk {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        if (!scanner.hasNextInt()) return;
        int n = scanner.nextInt();
        double grandTotal = 0.0;

        for (int i = 0; i < n; i++) {
            String type = scanner.next();
            double weight = scanner.nextDouble();
            double value = scanner.nextDouble();

            Parcel parcel = null;
            switch (type.toUpperCase()) {
                case "STANDARD": parcel = new StandardParcel(weight, value); break;
                case "EXPRESS": parcel = new ExpressParcel(weight, value); break;
                case "FRAGILE": parcel = new FragileParcel(weight, value); break;
            }

            if (parcel != null) {
                double charge = parcel.calculateCharge();
                double insurance = 0.0;
                if (parcel instanceof Insurable) {
                    insurance = ((Insurable) parcel).calculateInsurance(value);
                }
```

```
                double total = charge + insurance;
                grandTotal += total;
                System.out.printf("%s: Charge=%.2f Insurance=%.2f Total=%.2f
", type.toUpperCase(), charge, insurance, total);
            }
        }
        System.out.printf("Grand Total: %.2f
", grandTotal);
        scanner.close();
    }
}
```

Problem 3: College Fee Counter

Implements a Transportable interface for day scholars and scholarship students, separating bus fee computation from tuition structures.

Java Implementation:

```
import java.util.Scanner;

interface Transportable {
    double getTransportFee();
}

abstract class Student {
    protected String name;
    public Student(String name) { this.name = name; }
    public abstract double calculateTuition();
}

class DayScholar extends Student implements Transportable {
    public DayScholar(String name) { super(name); }
    @Override public double calculateTuition() { return 40000.0; }
    @Override public double getTransportFee() { return 12000.0; }
}

class Hosteller extends Student {
    public Hosteller(String name) { super(name); }
    @Override public double calculateTuition() { return 40000.0 + 60000.0; }
}

class ScholarStudent extends Student implements Transportable {
    public ScholarStudent(String name) { super(name); }
    @Override public double calculateTuition() { return 20000.0; }
    @Override public double getTransportFee() { return 12000.0; }
}

public class CollegeFeeCounter {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        if (!scanner.hasNextInt()) return;
        int n = scanner.nextInt();
        double totalCollected = 0.0;

        for (int i = 0; i < n; i++) {
            String type = scanner.next();
            String name = scanner.next();

            Student student = null;
            switch (type.toUpperCase().replace("_", "")) {
                case "DAYSCHOLAR": student = new DayScholar(name); break;
                case "HOSTELLER": student = new Hosteller(name); break;
                case "SCHOLAR": student = new ScholarStudent(name); break;
            }

            if (student != null) {
                double fee = student.calculateTuition();
                if (student instanceof Transportable) {
                    fee += ((Transportable) student).getTransportFee();
                }
                totalCollected += fee;
                System.out.printf("%s: %.2f
", student.name, fee);
            }
        }
        System.out.printf("Total Collected: %.2f
", totalCollected);
        scanner.close();
```

```
    }
}
```

Problem 4: City Cab Fare Meter

Utilizes an INightService interface for Sedans and SUVs, rejecting night trips for Mini cabs while enforcing minimum fare rules.

Java Implementation:

```
import java.util.Scanner;

interface INightService {
    boolean supportsNightService();
}

abstract class Cab {
    protected double km;
    protected String time;

    public Cab(double km, String time) {
        this.km = km;
        this.time = time;
    }

    public abstract double getRatePerKm();

    public double calculateFare() {
        double rawFare = km * getRatePerKm();
        double fare = Math.max(rawFare, 100.0);
        if (time.equalsIgnoreCase("NIGHT")) {
            fare *= 1.20;
        }
        return fare;
    }
}

class MiniCab extends Cab {
    public MiniCab(double km, String time) { super(km, time); }
    @Override public double getRatePerKm() { return 10.0; }
}

class SedanCab extends Cab implements INightService {
    public SedanCab(double km, String time) { super(km, time); }
    @Override public double getRatePerKm() { return 14.0; }
    @Override public boolean supportsNightService() { return true; }
}

class SuvCab extends Cab implements INightService {
    public SuvCab(double km, String time) { super(km, time); }
    @Override public double getRatePerKm() { return 18.0; }
    @Override public boolean supportsNightService() { return true; }
}

public class CityCabFareMeter {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        if (!scanner.hasNextInt()) return;
        int n = scanner.nextInt();
        double total = 0.0;

        for (int i = 0; i < n; i++) {
            String cabType = scanner.next();
            double km = scanner.nextDouble();
            String time = scanner.next();

            Cab cab = null;
            switch (cabType.toUpperCase()) {
                case "MINI": cab = new MiniCab(km, time); break;
                case "SEDAN": cab = new SedanCab(km, time); break;
                case "SUV": cab = new SuvCab(km, time); break;
```

```
            }

            if (cab != null) {
                if (time.equalsIgnoreCase("NIGHT") && !(cab instanceof INightService)) {
                    System.out.printf("%s: night service not available
", cabType.toUpperCase());
                } else {
                    double fare = cab.calculateFare();
                    total += fare;
                    System.out.printf("%s: %.2f
", cabType.toUpperCase(), fare);
                }
            }
        }
        System.out.printf("Total: %.2f
", total);
        scanner.close();
    }
}
```

Problem 5: Home Appliance Energy Report

Implements an ISaverMode interface for AC and Washer appliances to calculate discounted energy consumption.

Java Implementation:

```
import java.util.Scanner;

interface ISaverMode {
    boolean supportsSaver();
}

abstract class Appliance {
    protected double hours;
    protected boolean isSaver;

    public Appliance(double hours, boolean isSaver) {
        this.hours = hours;
        this.isSaver = isSaver;
    }

    public abstract double getPowerWatts();

    public double calculateUnits() {
        double units = (getPowerWatts() * hours) / 1000.0;
        if (isSaver) {
            units *= 0.75;
        }
        return units;
    }
}

class Fridge extends Appliance {
    public Fridge(double h, boolean s) { super(h, s); }
    @Override public double getPowerWatts() { return 150.0; }
}

class AirConditioner extends Appliance implements ISaverMode {
    public AirConditioner(double h, boolean s) { super(h, s); }
    @Override public double getPowerWatts() { return 1500.0; }
    @Override public boolean supportsSaver() { return true; }
}

class Television extends Appliance {
    public Television(double h, boolean s) { super(h, s); }
    @Override public double getPowerWatts() { return 100.0; }
}

class WashingMachine extends Appliance implements ISaverMode {
    public WashingMachine(double h, boolean s) { super(h, s); }
    @Override public double getPowerWatts() { return 500.0; }
    @Override public boolean supportsSaver() { return true; }
}

public class HomeApplianceEnergyReport {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        if (!scanner.hasNextInt()) return;
        int n = scanner.nextInt();
        double totalCost = 0.0;

        for (int i = 0; i < n; i++) {
            String name = scanner.next();
            double hours = scanner.nextDouble();
            boolean saver = false;
            if (scanner.hasNext("SAVER")) {
                saver = true;
```

```
                scanner.next();
            }

            Appliance app = null;
            switch (name.toUpperCase()) {
                case "FRIDGE": app = new Fridge(hours, saver); break;
                case "AC": app = new AirConditioner(hours, saver); break;
                case "TV": app = new Television(hours, saver); break;
                case "WASHER": app = new WashingMachine(hours, saver); break;
            }

            if (app != null) {
                if (saver && !(app instanceof ISaverMode)) {
                    System.out.printf("%s: saver mode not supported
", name.toUpperCase());
                } else {
                    double units = app.calculateUnits();
                    double cost = units * 8.0;
                    totalCost += cost;
                    System.out.printf("%s: Units=%.2f Cost=%.2f
", name.toUpperCase(), units, cost);
                }
            }
        }
        System.out.printf("Total Cost: %.2f
", totalCost);
        scanner.close();
    }
}
```